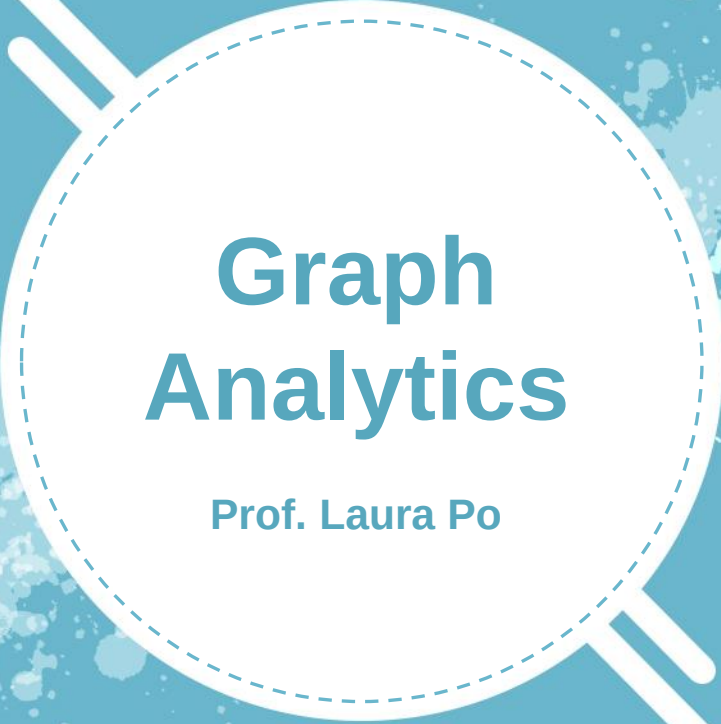




Graph Analytics

Prof. Laura Po

**Building
Recommendation
Systems
using Graphs**

Data modeling

We will focus on data modeling: the formal structures used to represent the training dataset and the inferred model (the result of the learning process) so that a computer program can process and access it to provide forecasting or analysis to end users.

Data modeling

Two models are taken into account:

- The **descriptive model** is a simplified representation of reality (the training dataset) created to serve a specific learning purpose. The simplification is based on some assumptions about what is and is not relevant for the specific purpose.
- The **predictive model** is a formula for estimating the unknown value of interest: the target. Such a formula could be mathematical, a query on a data structure (a database or a graph, for example), a logical statement such as a rule, or any combination of these. It represents, in an efficient format, the result of the learning process on the training dataset, and it is accessed to perform the actual prediction.

GOAL: provide you with the mental tools to represent the inputs and outputs of predictive techniques as graphs, showing the intrinsic value of the graph approach.



Recommendations

Recommender systems

- The term *recommender system* (RS) refers to all software tools and techniques that, by using the knowledge they can gather about the users and items in question, **suggest items that are likely to be of interest to a particular user** [Ricci et al., 2015].
- ***item*** is the general term used to identify what the system recommends to users. An RS normally focuses on a specific type or class of items, such as books to buy, news articles to read, or hotels to book. The overall design and the techniques used to generate the recommendations are customized to provide useful and relevant suggestions for that specific type of item.

**Why are recommendation systems
so commonly used?**



Main reasons

- *Increasing the number of items sold:* offering any kind of recommendation
- *Selling more diverse items:* allowing the user to select items that might be hard to find without a precise recommendation
- *Increasing user satisfaction:* If the user finds the recommendations interesting, relevant, they will enjoy using the system
- *Increasing user loyalty:* customer-centric applications appreciate loyalty by recognizing returning customers and treating them as valued visitors. Tracking returning users is a common requirement for RSs because the algorithms use the information acquired from users during previous interactions.
- *Getting a better understanding of what the user wants:* RS creates a model for the user's preferences, which are collected explicitly or predicted by the system itself.



Content-based Recommendations

OBJECTIVES

- Designing proper graph models for a content-based recommendation engine
- Importing existing (non-graph) datasets into the designed graph models
- Implementing working content-based recommendation engines

HAVE YOU EVER WORKED IN A SHOP
(OR RESTAURANT) FOR A WHILE?

HOW DID YOU INTERACT WITH CLIENTS?

Use case – Video rental store

Suppose you would like to build a movie recommender system for your local video rental store.

This scenario inherently has a lot in common with more-complex online recommender systems:

- A small user community
- A limited set of well-curated items: Each item (in this case, a movie) can have a lot of associated details. For movies, these details might include plot description, keywords, genres, and actors
- Knowledge of user preferences: The owner or shop assistant knows the preferences of almost all the customers, even if they've rented only a few movies or games.



Content-based recommender system (CBRS)

CBRSs rely on item and user descriptions (content) to build item representations (or item profiles) and user profiles to suggest items similar to those a target user liked in the past.

These types of recommender systems are also known as semantic-aware CBRs.

This approach allows the system to provide recommendations even when **the amount of data available is quite small** (that is, a limited number of users, items, or interactions).

Content-based recommender system (CBRS)

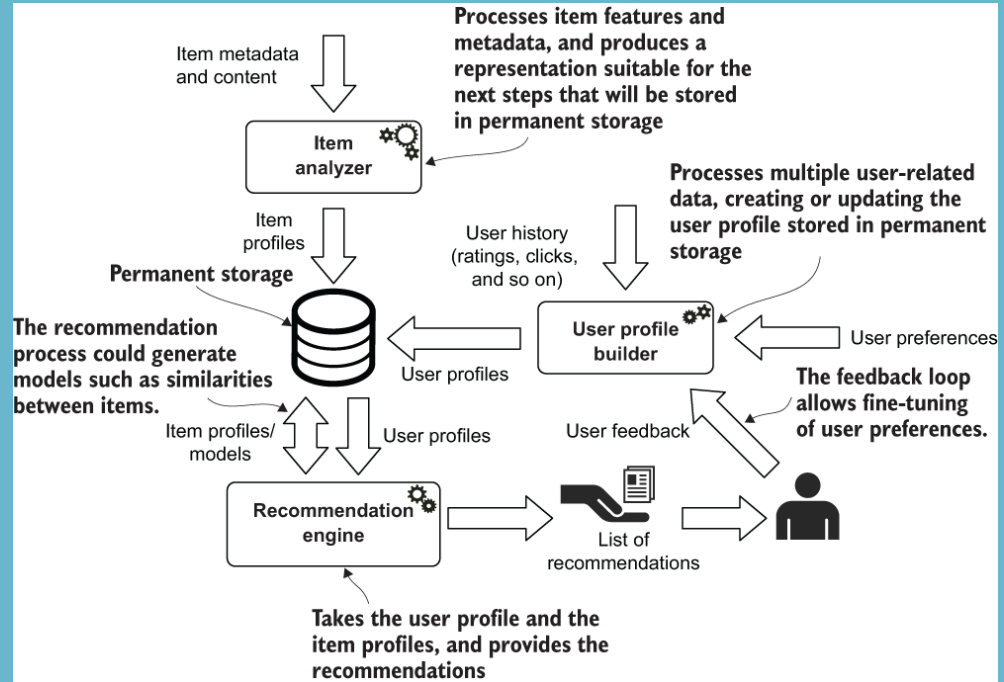
BASIC PROCESS

- matching the attributes of the target user profile, with attributes of the items to find items similar to what the user liked in the past
- the result is a relevance score that predicts the target user's level of interest in those items
- usually, attributes for describing an item are features extracted from metadata associated with that item or textual features somehow related to the item —descriptions, comments, keywords, and so on.
- CBRSs use these content-rich items for making comparisons or inferring a user's interests based on the list of items they've interacted with.

High level architecture

Three main components:

- *Item analyzer* - extract or identify relevant features (also called properties or attributes), and represent the items - takes as input the item content (contents of a book or a product description) and meta information (book's author, the actors in a movie...) from 1+ information sources and converts them to an item model that is used later to provide recommendations
- *User profile builder*—This process collects data representative of user preferences and infers user profiles - asking users about their interests or implicit feedback collected by observing and storing user behavior. The result is a model—specifically, a graph model—that represents user interest in some item, item feature, or both
- *Recommendation engine*—exploits the user profiles and item representations to suggest relevant items by matching user interests with item features. In this phase, a prediction model is built and used to create a relevancy score for each item for each user. This score is used to rank and order the items to suggest to the user.



Features

- Features are defined generally as **meta information** because they are not actually the content of the item. Unfortunately, there are classes of items for which it is not so easy to find or identify features, such as document collections, email messages, news articles, and images.
- Text-based items do not tend to have readily available sets of features. Nonetheless, their content can be represented by identifying a set of features that describe them. A common approach is the identification of words that characterize the topic. Different techniques exist for accomplishing this task. The result is a list of features (keywords, tags, relevant words) that describe the content of the item. These features can be used to represent a text-based item in exactly the same way as the meta information.

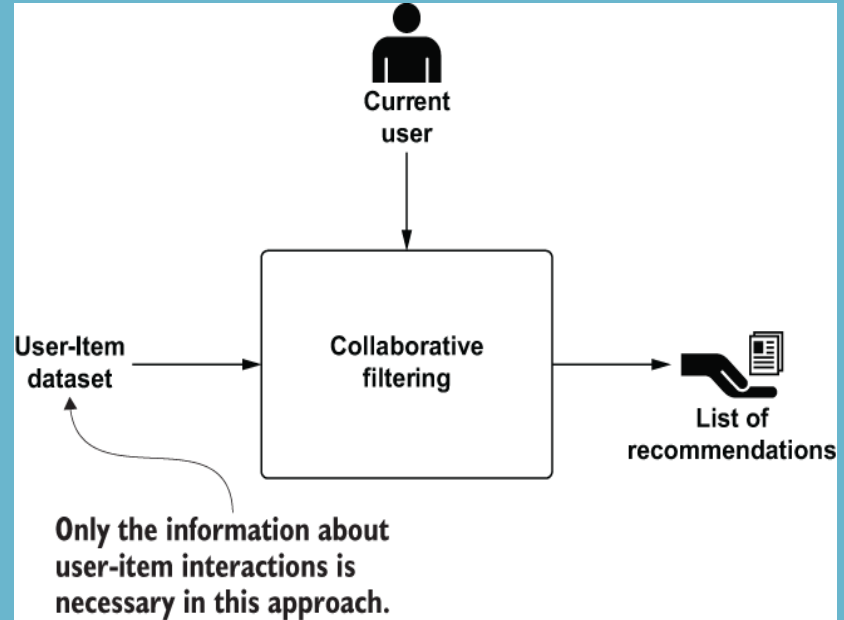
Users

- In a CBRS, several methods exist for gathering and modeling user profiles.
- The selected design model will vary according to how the preferences are collected (implicitly or explicitly) and the type of filtering strategy or recommendation approach.
- A straightforward way to collect user preferences is to ask the user. The user might express interest in specific genres or keywords, or in particular actors or directors.
- From a high-level perspective, the purpose of the user profile and the defined model is to help the recommendation engine assign a score to each item or item feature.
- The score helps rank the items suggested to the specific user, ordered from high to low. For this reason, recommender systems belong to the area of machine learning called *learning to rank*.

**Collaborative
Filtering
Recommendations**

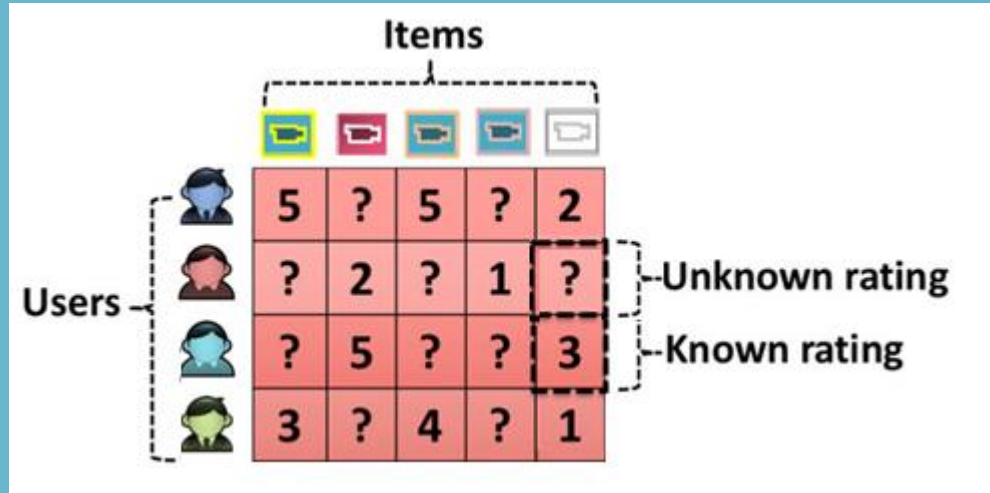
Collaborative Filtering

- An alternative to content filtering relies only on past user behavior, such as previous transactions or item ratings, or the opinions of an existing user community to predict which items the users will most probably like or be interested in without requiring the creation of explicit profiles for both items and users based on item features.
- This approach is known as *collaborative filtering*, a term coined by the developers of Tapestry [Goldberg et al., 1992], the first recommender system. Collaborative filtering analyzes relationships between users and interdependencies among items to predict new user-item associations.



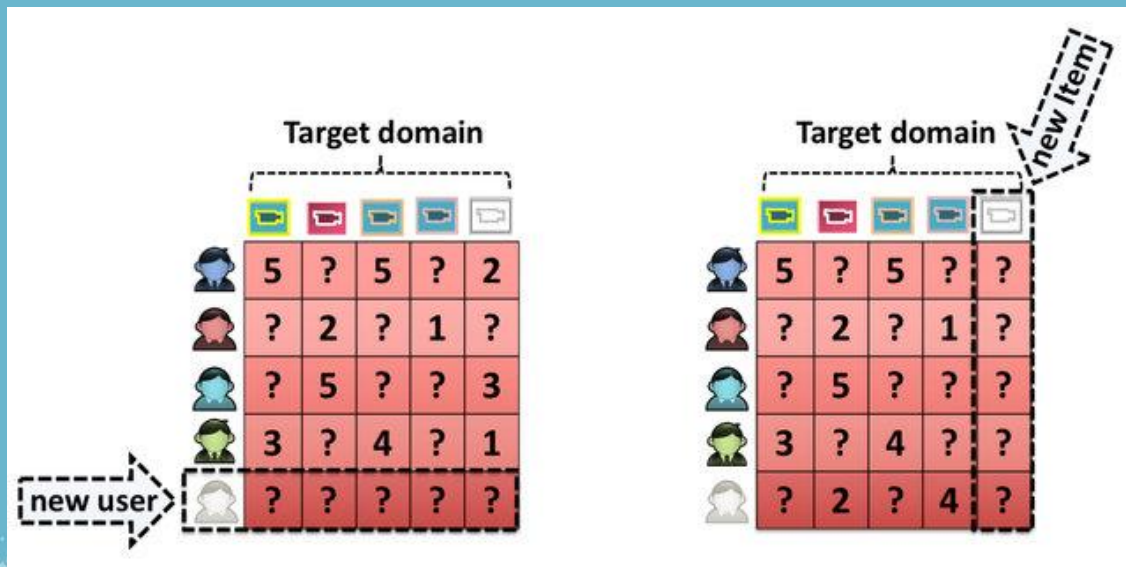
Advantages

- it is domain free
- it doesn't require any detail about the items
- It can be applied to a vast variety of use cases and scenarios, and it can address data aspects that are often elusive and difficult to profile by using content filtering.



Disadvantages

- The *cold-start problem* is due to its inability to provide reasonable (in terms of accuracy) recommendations for new items and users or when limited interaction data is available.
- Nonetheless, mechanisms do exist that mitigate the effect of the cold-start problem by using different algorithms, such as the graph approach or other sources of knowledge, such as social networks.



Two main approaches

- *Memory-based*—Memory-based supposes that if a user likes the movie *Saving Private Ryan*, they may like similar movies, such as war movies, Spielberg movies, and Tom Hanks movies [Koren et al., 2009].
- Also called neighborhood-based methods
- To predict a particular user's rating for *Saving Private Ryan*, we would look for the movie's nearest neighbors that this user rated. Alternatively, the algorithm can look for similar users based on the set of films they watched and suggest something that the current user hasn't watched yet. In these methods, the User-Item dataset that is stored is used directly to predict ratings for items.

Two main approaches

- *Model-based*—create models for users and items that describe their behavior via a set of factors or features and the weight these features have for each item and each user. In the movie example, the discovered factors might measure obvious dimensions such as genre (comedy, drama, action) or orientation to children; less well-defined dimensions such as depth of character development or quirkiness; or uninterpretable dimensions. For users, each factor expresses how much the user likes movies that score high on the corresponding factor. In these methods, the raw data (the User-Item dataset) is first processed offline, with the information on ratings or previous purchases used to create this predictive model. At run time, only the precomputed or learned model is required to make predictions during the recommendation process. Latent factor models represent the most common approaches in this class. They try to explain the ratings by characterizing both items and users on, say, 20 to 100 factors inferred from the rating patterns. In a sense, such factors comprise a computerized alternative to the human-created features encountered in the content-based recommendation systems.



Thank you

Insert the title of your subtitle Here